

FluCoMa

Programming and Music 2
<http://www.bjarni-gunnarsson.net>

FluCoMa

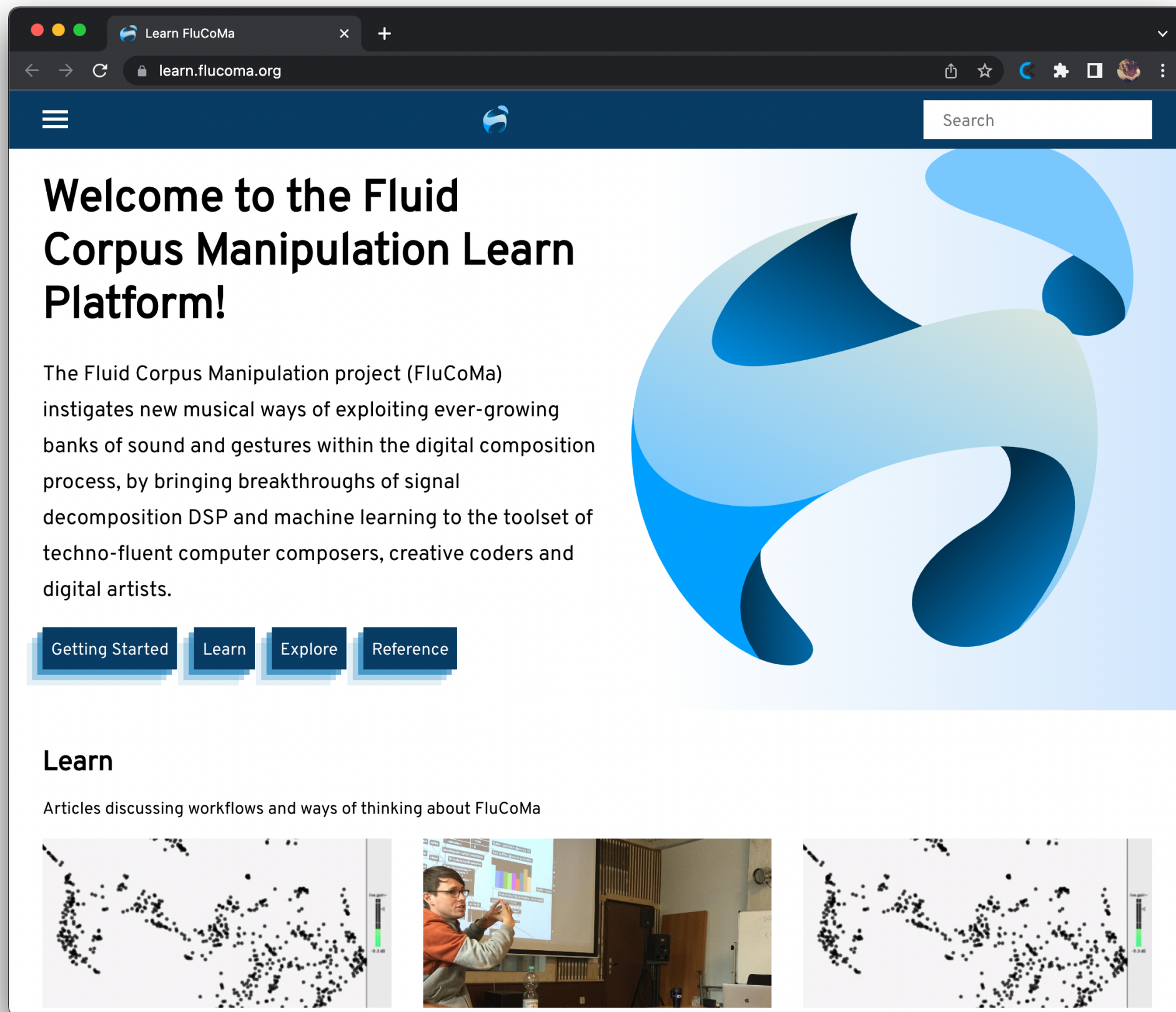
FluCoMa

Mission: *“To enable techno-fluent musicians to use machine listening and machine learning in their creative practices ”*

FluCoMa provides a toolset for ***signal decomposition***, and a toolset for ***machine learning***, as well as support material, in order to experiment with new sound and gesture design untapped in large corpora from within their high-level creative coding workflow.

FluCoMa is based within the Department of Music and Music Technology, with its Centre for Research in New Music, on the Queensgate Campus of the University of Huddersfield.

FluCoMa



FluCoMa

f.scd (~/Desktop) - SuperCollider IDE

f.scd

```
1 ~src = Buffer.read(s,FluidFilePath("Tremblay-ASWINE-ScratchySynth-M.wav"));
2
3 // use .fromBuffer to add *every* mfcc analysis to the dataset
4 // pass a labelset so the identifiers aren't just numbers
5 (
6 fork{
7   var mfccs = Buffer(s);
8   FluidBufMFCC.processBlocking(s,~src,features:mfccs);
9   s.sync;
10  ~ls = FluidLabelSet(s);
11  mfccs.numFrames.do{
12    arg i;
13    // in the label set that gets used to create
14    // the identifiers, the identifier is the
15    // frame number in the buffer (an integer)
16    // and the "label" is what will be come the
17    // identifier in the dataset
18    ~ls.addLabel(i,"ASWINE-mfcc-%".format(i));
19  };
20  ~ds = FluidDataSet(s).fromBuffer(mfccs,0,~ls).print;
21 }
22 )
23
24 // create a small DataSet...
25 f = FluidDataSet(s)
26 // and fill it with a grid of data
27 f.load(Dictionary.newFrom(["cols", 2, "data", Dictionary.newFrom(9.collect{|i|["item-%".format(i), [i.div(3),
28 i.mod(3)] / 2}].flatten(1))]))
29 // the data looks like this
30 // (item-0 -> [ 0.0, 0.0 ]) (item-1 -> [ 0.0, 0.5 ]) (item-2 -> [ 0.0, 1.0 ])
31 // (item-3 -> [ 0.5, 0.0 ]) (item-4 -> [ 0.5, 0.5 ]) (item-5 -> [ 0.5, 1.0 ])
32 // (item-6 -> [ 1.0, 0.0 ]) (item-7 -> [ 1.0, 0.5 ]) (item-8 -> [ 1.0, 1.0 ])
33
34 // create a query buffer...
35 b = Buffer.alloc(s,2)
36
37 // and fill it with a point
38 b.sendCollection([1,0]);
39
40 // and request 9 nearest neighbours
41 f.kNearest(b,9,{|x|x.postln;})
42 he
```

Help browser Loading.. Home getn

Table of Contents

Usage

DataSet

An associative data container

Sound Into Sound

Exploring the Oboe with FluCoMa

Levels of Translation

Interface Design with FluCoMa

Batch Processing with FluCoMa

Neural Network Training

Comparing Scalers

Distribution and Histograms

Outliers

Smoothing Data

BufFlatten

DataSetQuery

Grid

KDTree

KNNClassifier

KNNRegressor

LabelSet

MDS

MLPClassifier

MLPRegressor

Normalize

PCA

Plotter

RobustScale

Standardize

The [DataSet](#) is an important object in the second FluCoMa toolbox. Whenever there is a need to create a collection of data (such as for input to a machine learning process), or to make a provision for some output, [DataSet](#) is the tool that you'll want to reach for.

The [DataSet](#) starts out empty and we add any number of *points* to it. A point is made up of two bits of information: an *identifier* and some *data*. The *data* is any number of numerical bits of information stored in a buffer. The *identifier* is a string or symbol which is *associated* to the *data*. If you have used the `coll`, `dict`, `text` or `Dictionary` objects in your environment of choice you will already be familiar with this idea of storing data associatively. If you're not, you can think of it like a filing system. The identifier gives us a human-readable bit of information which we can use to look up the data that it is attached to.

A small [DataSet](#) is depicted below, with 5 points. Each identifier is an instrument name, and we might imagine the data associated to each of these identifiers could be descriptor values or parameters for example.

Post window Help browser

Interpreter: Active Server: 0.08% 0.22% 0u 0s 2g 181d 0.0dB M R

FluCoMa

Analyse Data

BufStats
DataSet
DataSetQuery
Grid
KDTree
KMeans
KNNClassifier
KNNRegressor
LabelSet
MDS
MLPClassifier
MLPRegressor
Normalize
PCA
Plotter
RobustScale
SKMeans
Standardize
Stats
UMAP

Analyse Sound

AmpFeature
BufNMFSeed
BufSTFT
Chroma
Loudness
MelBands
MFCC
NMFMatch
NoveltyFeature
OnsetFeature
Pitch
SineFeature
SpectralShape

Utility Objects

BufCompose
BufFlatten
BufScale
BufSelect
BufSelectEvery
BufThresh

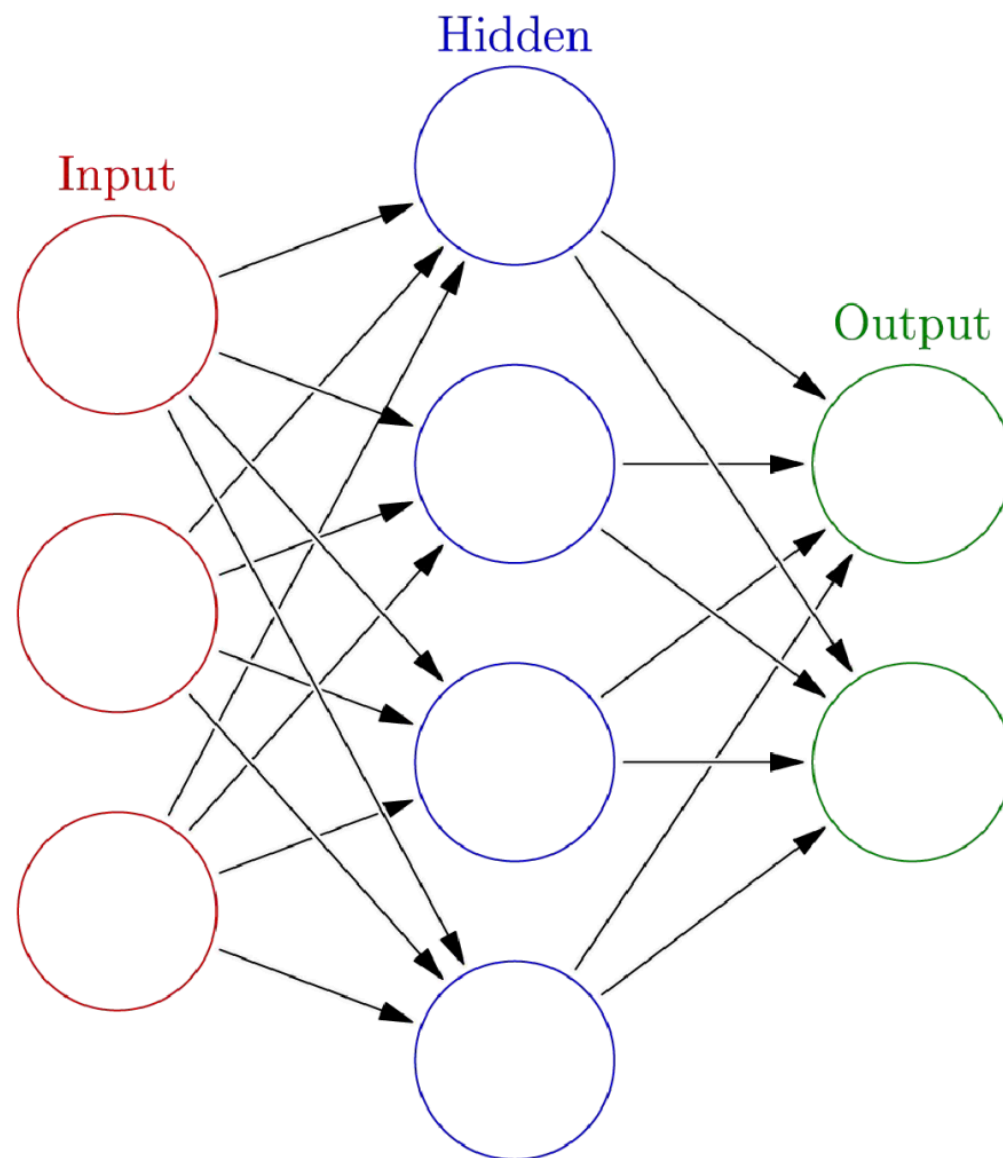
Decompose Sound

BufNMF
HPSS
Sines
Transients

Slice Sound

AmpGate
AmpSlice
NoveltySlice
OnsetSlice
TransientSlice

Neural Network



Regression:

a neural network predicts

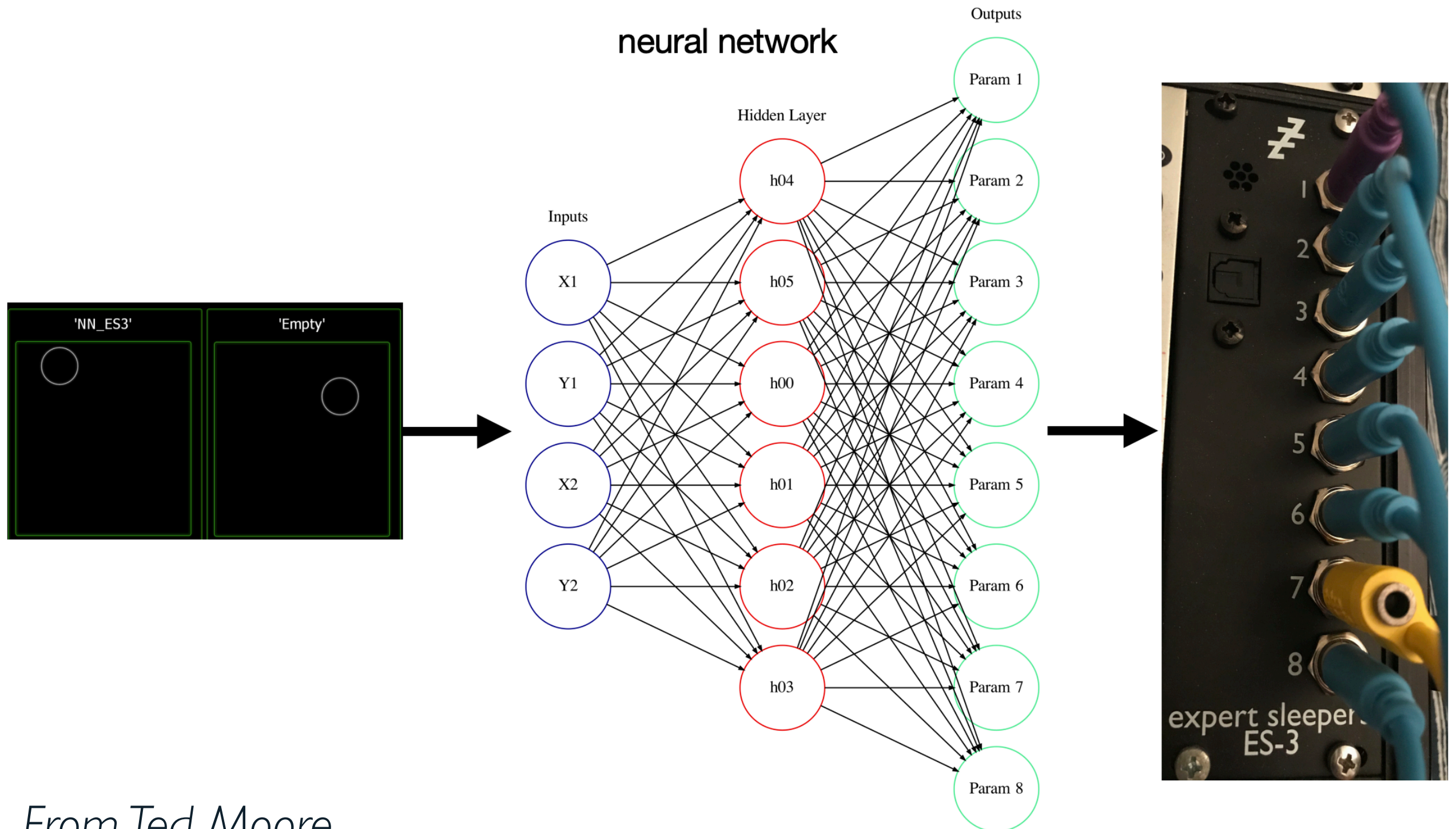
output values (to anything)

from

input values (from anything)

From Ted Moore

FluCoMa



From Ted Moore

Bias (2020) // for bass clarinet and interactive music system

A

Artemi-Maria Gioti: Bias

Watch Later

Share

More videos

Play (k)

0:09 / 6:31

YouTube



Table of Contents

[The Podcast](#)[Overview](#)[Links](#)

FluCoMa Podcast #12: Artemi-Maria Gioti

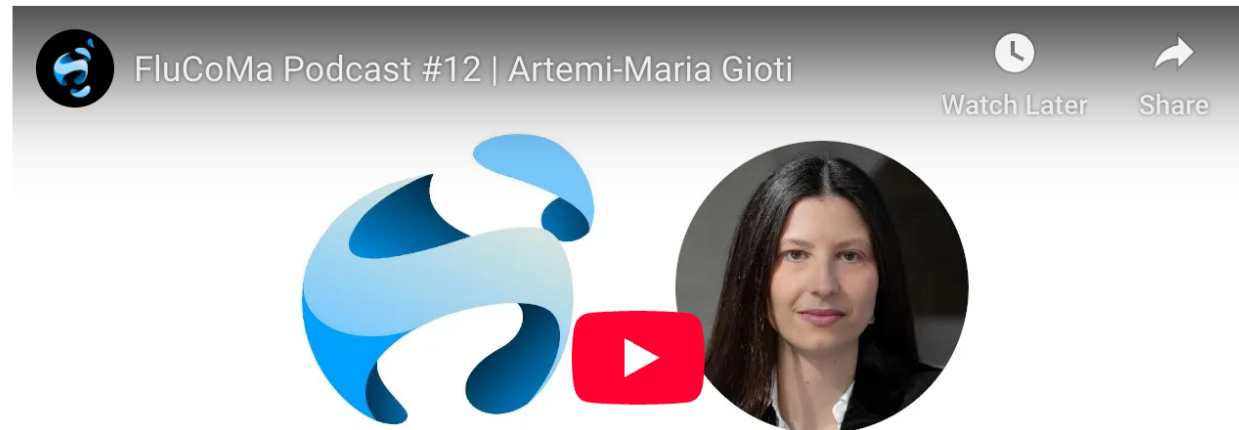
by [Jacob Hart](#)

Episode twelve of the FluCoMa podcast where Jacob Hart talks with composer, researcher and creative coder Artemi-Maria Gioti.



[Artemi-Maria Gioti](#)

The Podcast



FluCoMa Podcast

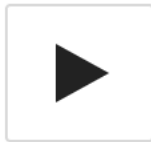
Watch on YouTube

Chapters

[1. Introduction \(0:0\)](#)

Extension

by [James Bradbury](#)



Process 1 01:21 / 04:24



Digital Album

Streaming + Download

Includes unlimited streaming via the free Bandcamp app, plus high-quality download in MP3, FLAC and more.

Buy Digital Album €4.99 EUR or more

[Send as Gift](#)

Subscription

Subscribe now to receive all the new music SUPERPANG releases, including this album and 187 back-catalog items, delivered instantly to you via the Bandcamp app for iOS and Android. You'll also get access to subscriber-only exclusives. [Learn more.](#)

Subscribe Now €9 EUR/month or more

- ▶ 1. [Process 1](#) 04:24
- ▶ 2. [Process 2](#) 01:47
- ▶ 3. [Process 3](#) 01:17
- ▶ 4. [Process 4](#) 04:45
- ▶ 5. [Process 5](#) 01:27
- ▶ 6. [Process 6](#) 03:33
- ▶ 7. [Process 7](#) 01:03
- ▶ 8. [Process 8](#) 03:24

Extension is a collection of responses to the two pieces in my previous album Interferences

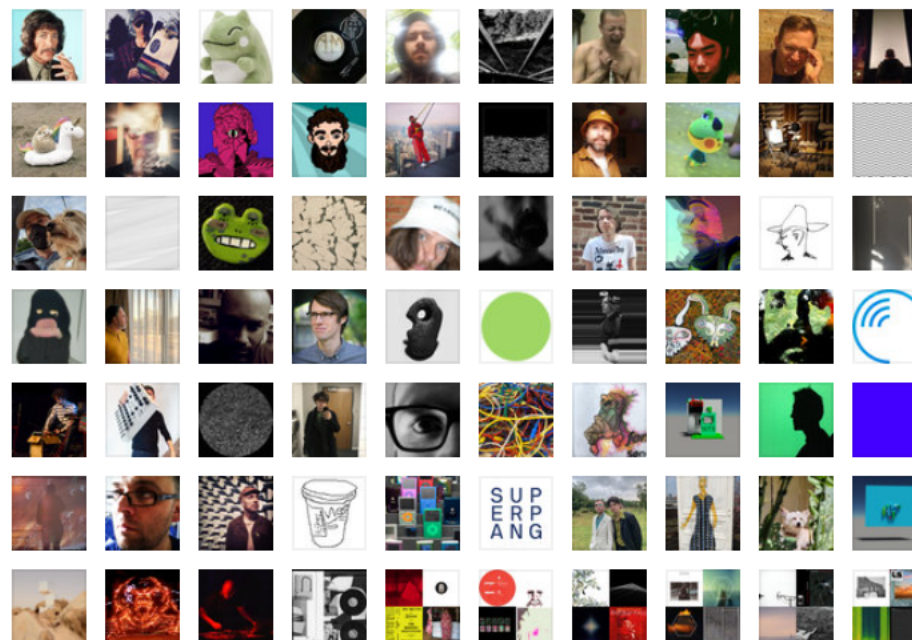
SP135

James Bradbury, Extension

[Share / Embed](#)

[Wishlist](#)

supported by



James Bradbury

Huddersfield, UK

[Follow](#)

artist from perth, wa who currently lives in berlin. I make electronic music with the help of my good friend the computer.

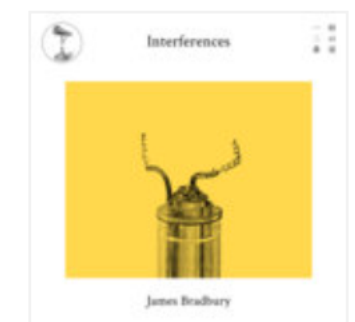
[jamesbradbury.net](#)

discography



[Extension](#)

Oct 2022



[Interferences](#)

Apr 2022

SP135

James Bradbury, Extension

Concepts

FluCoMa

Fluid Corpus Manipulation — data-driven music making in SuperCollider

- A toolkit for **analysing**, **storing**, and **querying** sound in SuperCollider, Max, and Pure Data.
- Works on **two timescales**: real-time (`.kr/``.ar`) and offline on whole buffers (`.processBlocking`).
- Maps audio to a **feature space**: each slice becomes a point with descriptors like brightness, loudness, MFCC.
- Provides **data structures** (`FluidDataSet`, `FluidLabelSet`, `FluidKDTree`) to organise points and look them up.
- Plus **creative-decomposition** tools — split a sound into harmonic / percussive / sinusoidal / transient / NMF components.
- Class 1 covers the **analysis half**: atomic objects → composing them into a corpus pipeline.
- Class 2 adds machine learning on top (classifiers, regressors, dim-reduction).

Real-time descriptors

Streaming analysis at control rate — feed into synths or capture per-frame

- `FluidLoudness.kr` — **LUFS + truePeak**. Perceptually-weighted loudness, EBU R128 spec.
- `FluidPitch.kr` — **[Hz, confidence 0..1]**. 4 algorithms; default YinFFT.
- `FluidMFCC.kr` — **13-dim timbre vector** per frame; drop coeff 0 for loudness-invariance.
- `FluidSpectralShape.kr` — **7 descriptors at once**: centroid, spread, skewness, kurtosis, rolloff, flatness, crest.
- `FluidOnsetSlice.ar` — sample-accurate **trigger** on each detected onset.
- `FluidAmpFeature.ar` — **fast/slow envelope differential** in dB; positive on transients.
- `FluidKrToBuf.kr(stream, buf)` — write the live `kr` stream into a 1-frame buffer so other FluCoMa objects can read it without OSC round-trip.

Offline / buffer descriptors

Same family, but on an entire pre-loaded buffer; emit a feature buffer

- `FluidBufLoudness`, `FluidBufPitch`, `FluidBufMFCC`, `FluidBufSpectralShape` — multi-channel feature buffers; one column per analysis frame.
- Call with `.processBlocking` for short jobs (runs in the server command queue, in order).
- `FluidBufStats` collapses a feature time-series to **7 statistics per channel**: *mean, std, skewness, kurtosis, low-percentile, mid-percentile, high-percentile*.
- `FluidWaveform(src, featuresBuffer:, stackFeatures: true)` draws each feature channel as a strip under the waveform — instant sanity check.
- Pipeline per slice: **analyse** → **BufStats** → **BufCompose into a fixed-size point**.
- Use `FluidBufCompose` (not `FluidBufFlatten`) when you need to write feature A into frame 0 and feature B into frame 1 — Flatten has no `destStartFrame`.
- Output points go into a `FluidDataSet` (server-side table of id → vector).

Slicers, chopping a buffer into events

Four algorithms; each catches a different kind of “character”

- `FluidBufOnsetSlice` — **spectral flux peaks**; finds drum hits, plucks, articulated attacks.
- `FluidBufNoveltySlice` — **spectral self-similarity**; catches timbre changes that aren't necessarily loud.
- `FluidBufAmpSlice` — **amplitude envelope crossings** through a threshold; classic gate logic.
- `FluidBufTransientSlice` — **LPC-based, sample-accurate** spike detection (Godsill / Rayner).
- All write a 1-channel "indices" buffer of **sample positions** where slices start.
- Pick the one that matches your material: percussion → Onset/Transient; texture → Novelty; speech → Amp.
- Pair with `FluidWaveform(src, slicesBuffer)` to see the slice points on the waveform before committing.

Decomposition, pulling sounds apart

Four ways to split one sound into multiple components that null-sum back

- `FluidBufHPSS` — **harmonic vs percussive**, median-filter the spectrogram (horizontal = harmonic, vertical = percussive).
- `FluidBufSines` — **sinusoidal partials vs residual**, peak-tracking across frames; tonal vs noise.
- `FluidBufTransients` — **sample-accurate spikes vs smooth residual**, LPC prediction error.
- `FluidBufNMF` — **N spectral templates** (bases $\times$ activations $\approx$ spectrogram); set `components: N` for N output channels.
- Each produces multi-output buffers; sum them back and you recover the original (to a good approximation).

2D sound browser

From feature space to playable interface in five steps

- **1) Slice** the source with `FluidBufOnsetSlice`.
- **2) Describe** each slice with `FluidBufSpectralShape` + `FluidBufLoudness`, summarised via `FluidBufStats` → 2-D point `[centroid_mean, loudness_mean]`.
- **3) Normalise.** `FluidNormalize(0, 1)` rescales both axes to `0..1` so neither dominates distance. *"Scale before lookup"* — the headline lesson.
- **4) Index.** `FluidKDTree.fit(~dsN)` builds a tree for fast nearest-neighbour search in feature space.
- **5) Browse.** `FluidPlotter` draws the scatter; `mouseMoveAction` sends `(x, y)` into `tree.kNearest` → look up id in `~map` → play that slice.

Cross-synthesis & compositional decomposition

- **NMFCross** (`FluidBufNMFCross`) — hybridise the *timbre* of a source with the *rhythm/activations* of a target. "Speech → played on a violin texture."
- Parameters that shape character: `timeSparsity`, `polyphony`, `continuity`, `windowSize`.
- **Compositional decomposition**: split a source into harmonic + percussive + transient, drive each part with its own `Pdef` — re-mix live.
- Each component becomes a **redefinable Pdef surface**; the corpus instrument is a tiny live-coding patch.

Exercises

Exercises

1. Compare two pitch algorithms atomic/realtime-descriptors.scd

In the FluidPitch.kr block on melodie.wav, swap algorithm: 2 (YinFFT) for algorithm: 3 (Yin). Which gives steadier pitch and higher confidence?

2. Compare two MFCC vectors atomic/buffer-descriptors.scd

Run the MFCC + loadToFloatArray block on ~texture, then on ~vocal. Which coefficient differs most? That's where the timbral difference lives.

3. Onset vs Novelty on the same buffer atomic/slicers.scd

Slice branches.wav with FluidBufOnsetSlice, then with FluidBufNoveltySlice. Compare slice counts and overlay both with FluidWaveform.

Exercises

4. NMF: choose components by ear atomic/decomposition.scd

Change components: 3 to 5 and extend the SynthDef to 5 channels. Solo each. Do any two sound identical? Drop back. Does one sound like "everything"?

5. Build a corpus from your own folder methods/corpus-building.scd

Point ~sounds at a folder with 3–5 of your own short .wav files and run. Does ~ds.print's point count match the slices you'd expect by ear?

6. Swap loudness → pitch on the Y axis methods/sound-browser.scd

Replace the loudness branch with FluidBufPitch in the 2-D analysis. Re-run analysis, normalisation, KDTree, mouse block. Does dragging feel more or less musical now?

